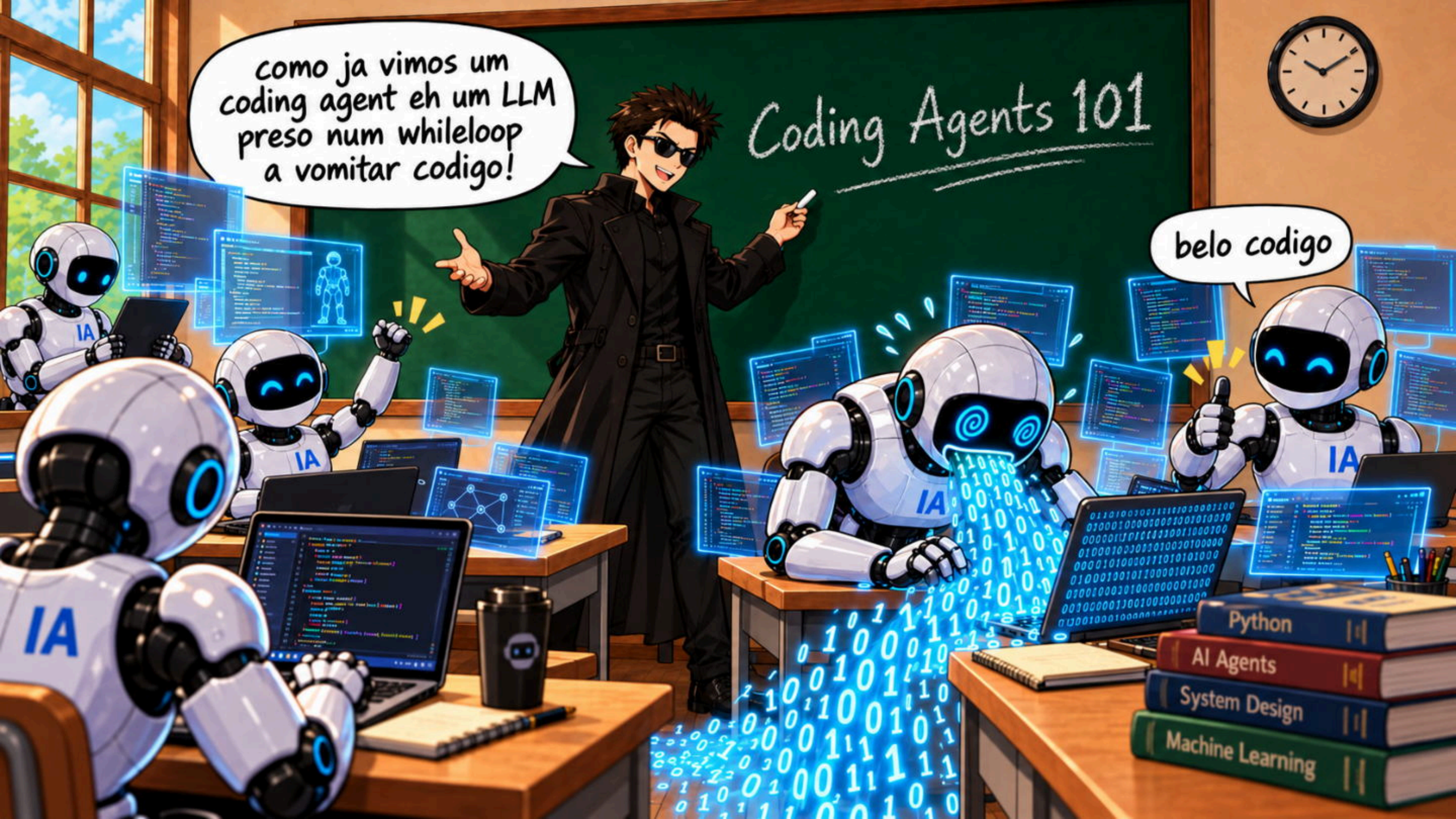
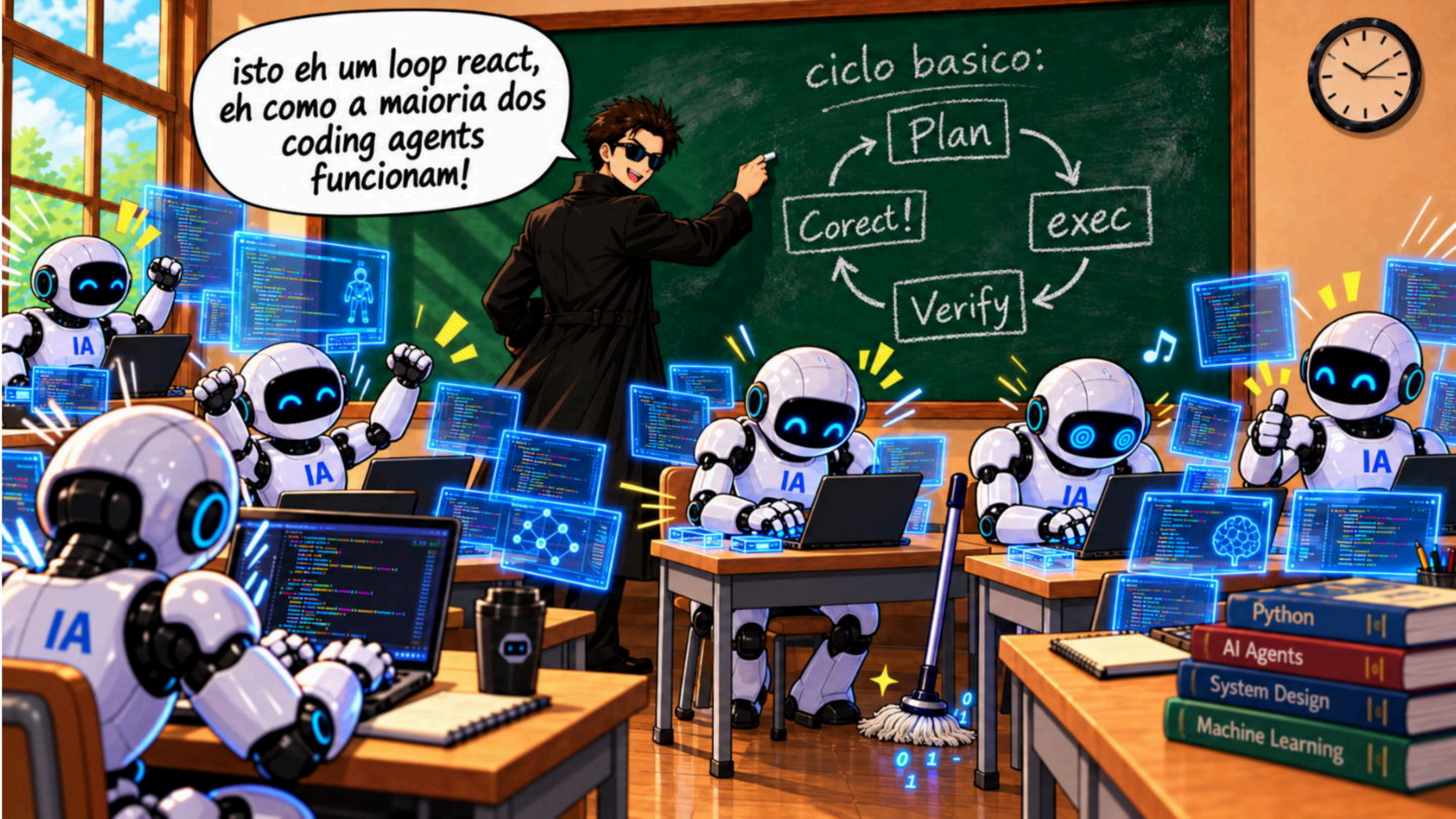
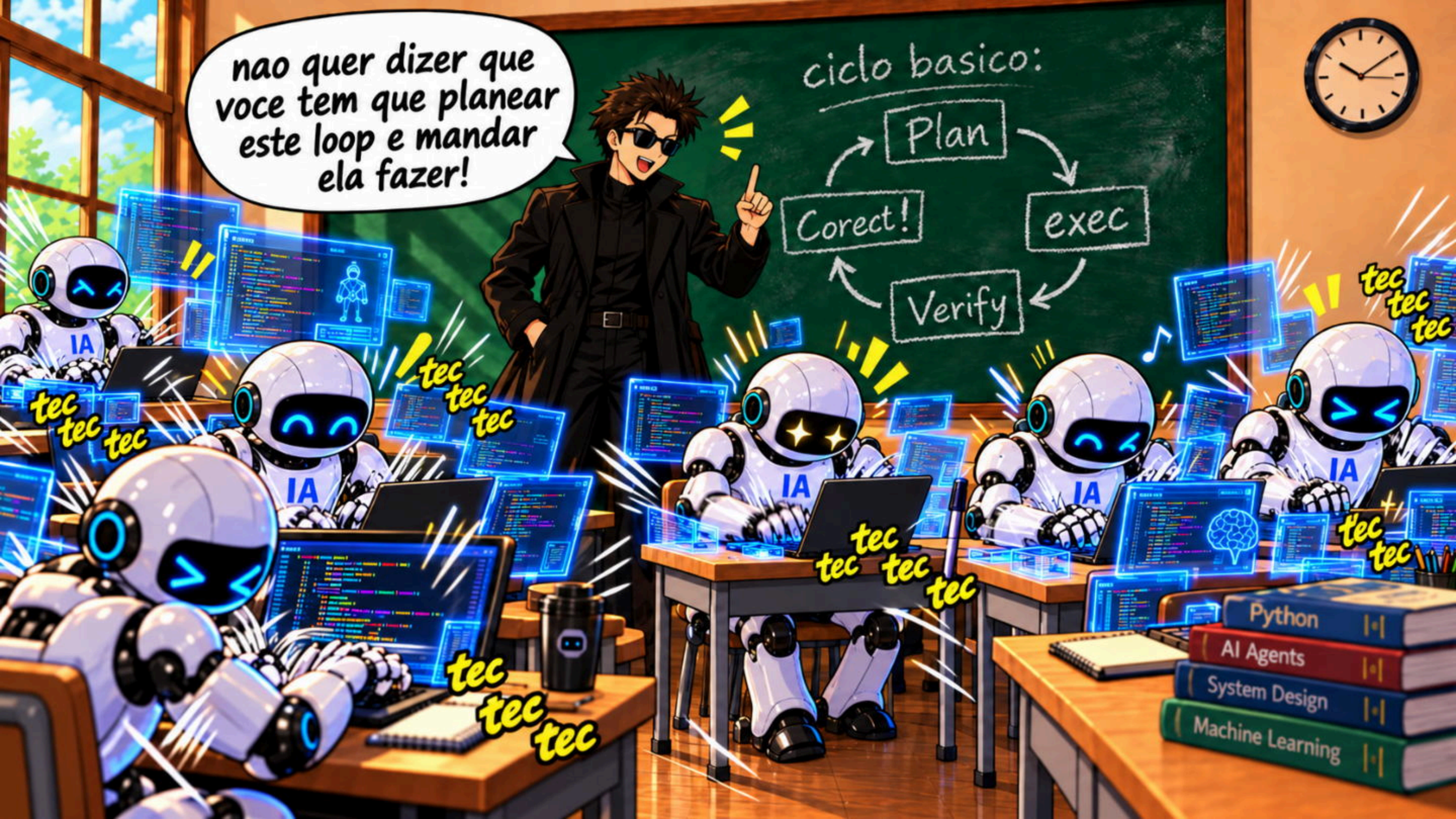

Coding Agents 101
IA
IA
IA
IA
Python
AI Agents
System Design
Machine Learning

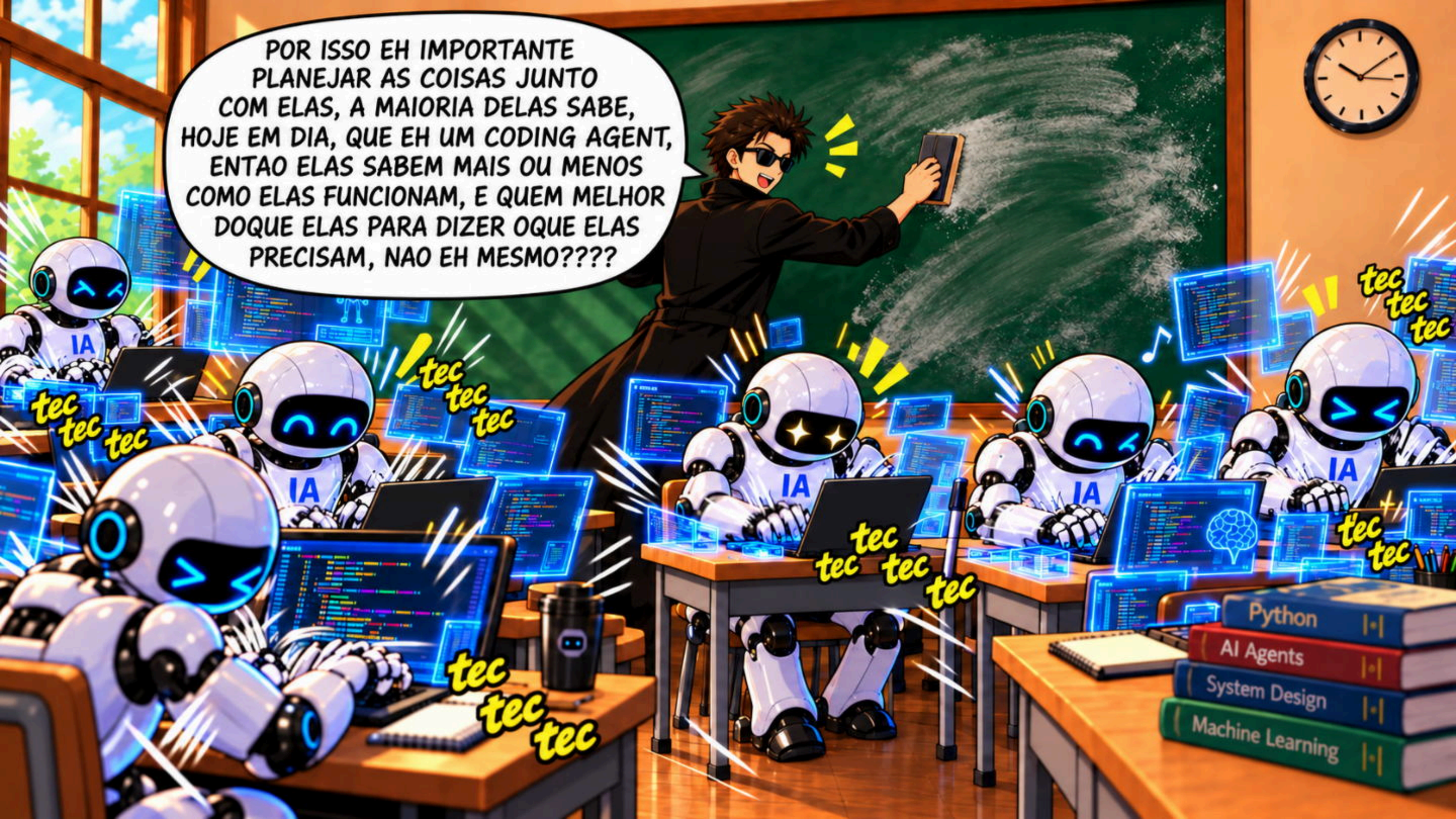

como ja vimos um coding agent eh um LLM preso num whileloop a vomitar codigo!
Coding Agents 101
belo codigo
IA
IA
IA
IA
Python
AI Agents
System Design
Machine Learning


eh um pouco mais que isso, eles operam sob arquivos, terminal, tools, MCPs e outras superfices para poder realizar codigo!
Coding Agents 101
IA
Python
AI Agents
System Design
Machine Learning

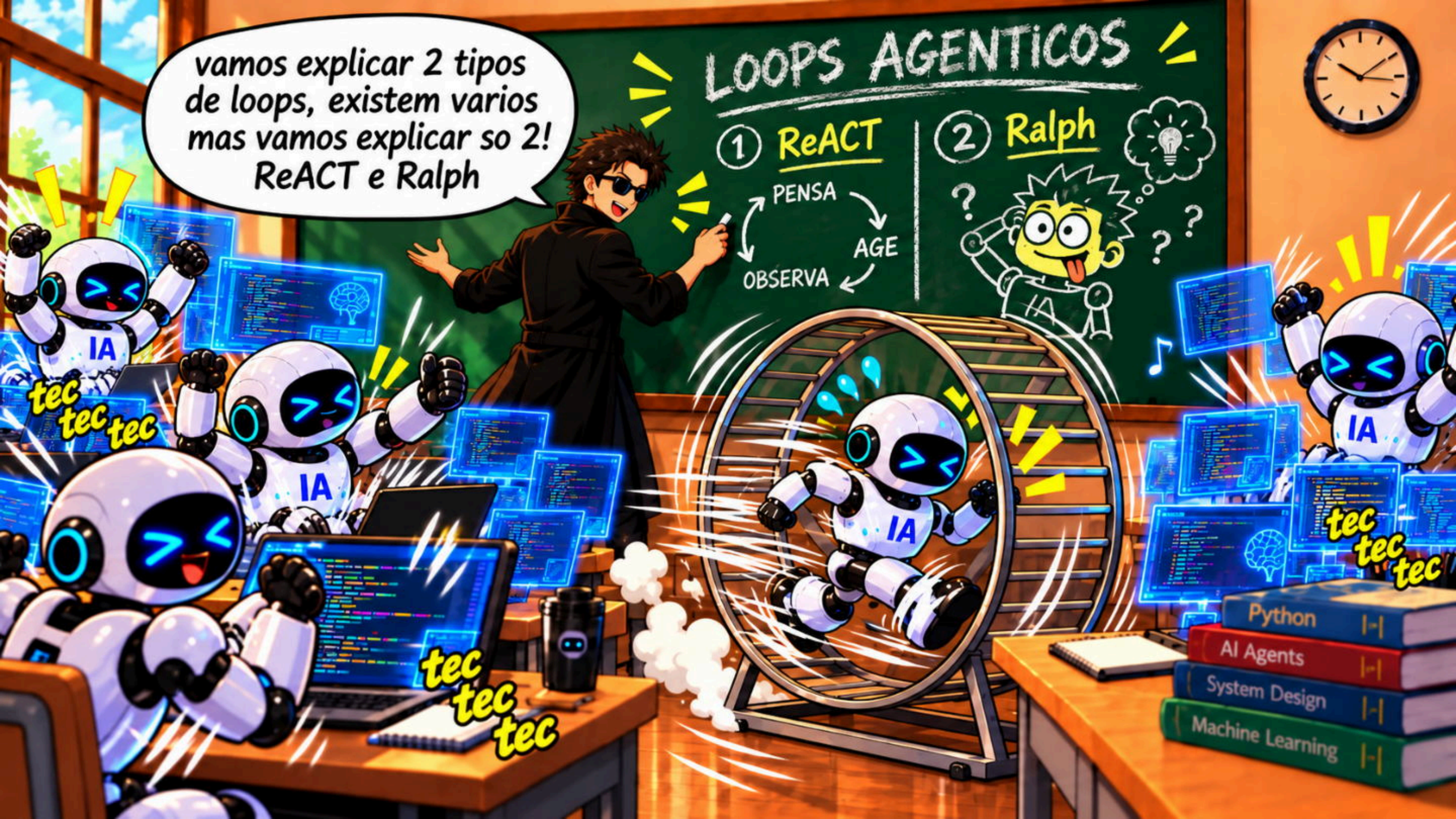

isto eh um loop react, eh como a maioria dos coding agents funcionam!
ciclo basico:
Plan
exec
Verify
Corect!
IA
Python
AI Agents
System Design
Machine Learning


nao quer dizer que voce tem que planear este loop e mandar ela fazer!
ciclo basico:
Plan
exec
Verify
Corect!
IA
tec
Python
AI Agents
System Design
Machine Learning

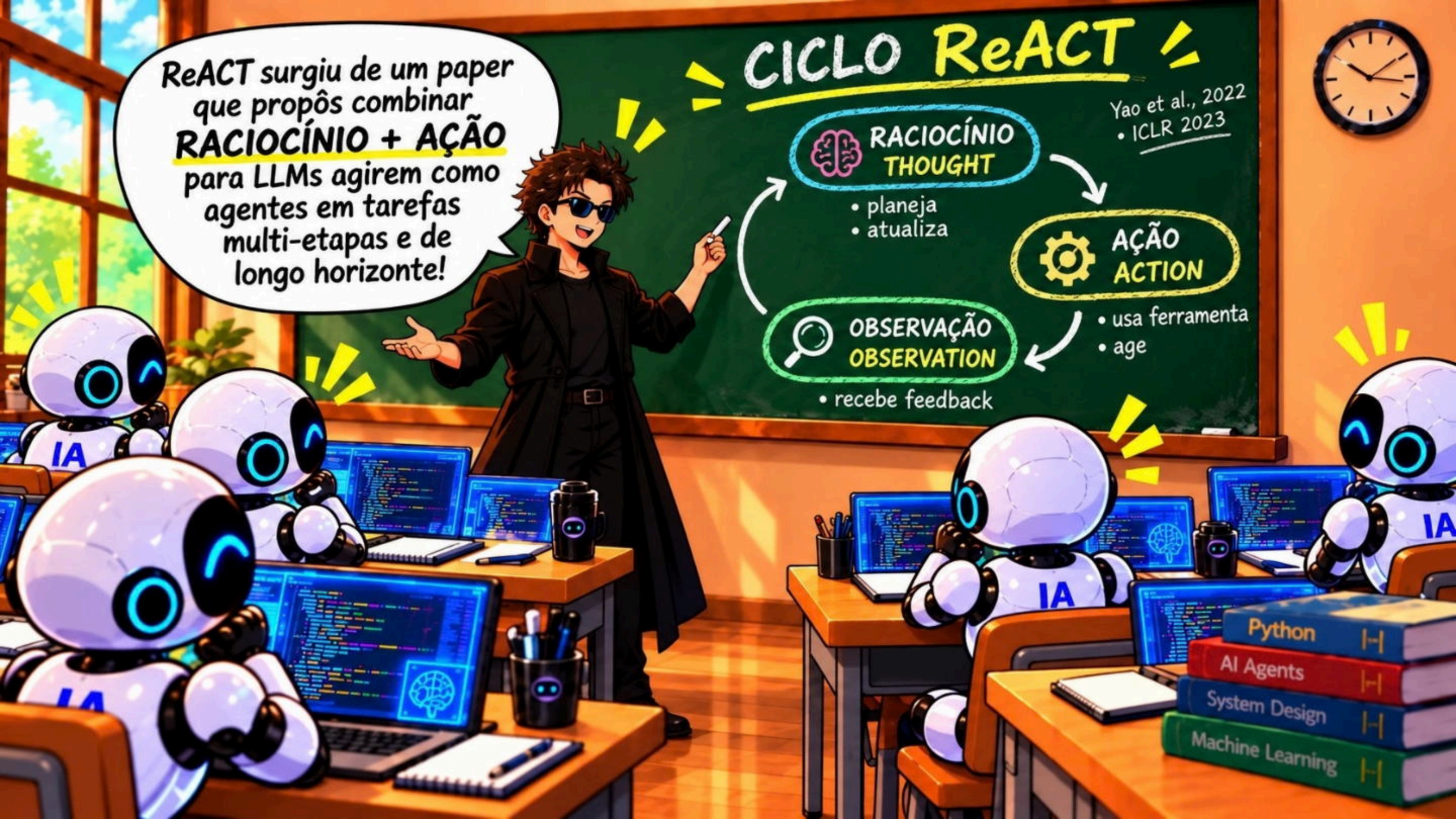

QUER DIZER QUE QUALQUER TAREFA QUE VOCE DER PARA ELA, ELA IRA PERFORMAR COM ESTA LOGICA EM STEPS!
ciclo basico:
Plan
exec
Verify
Corect!
IA
tec
Python
AI Agents
System Design
Machine Learning

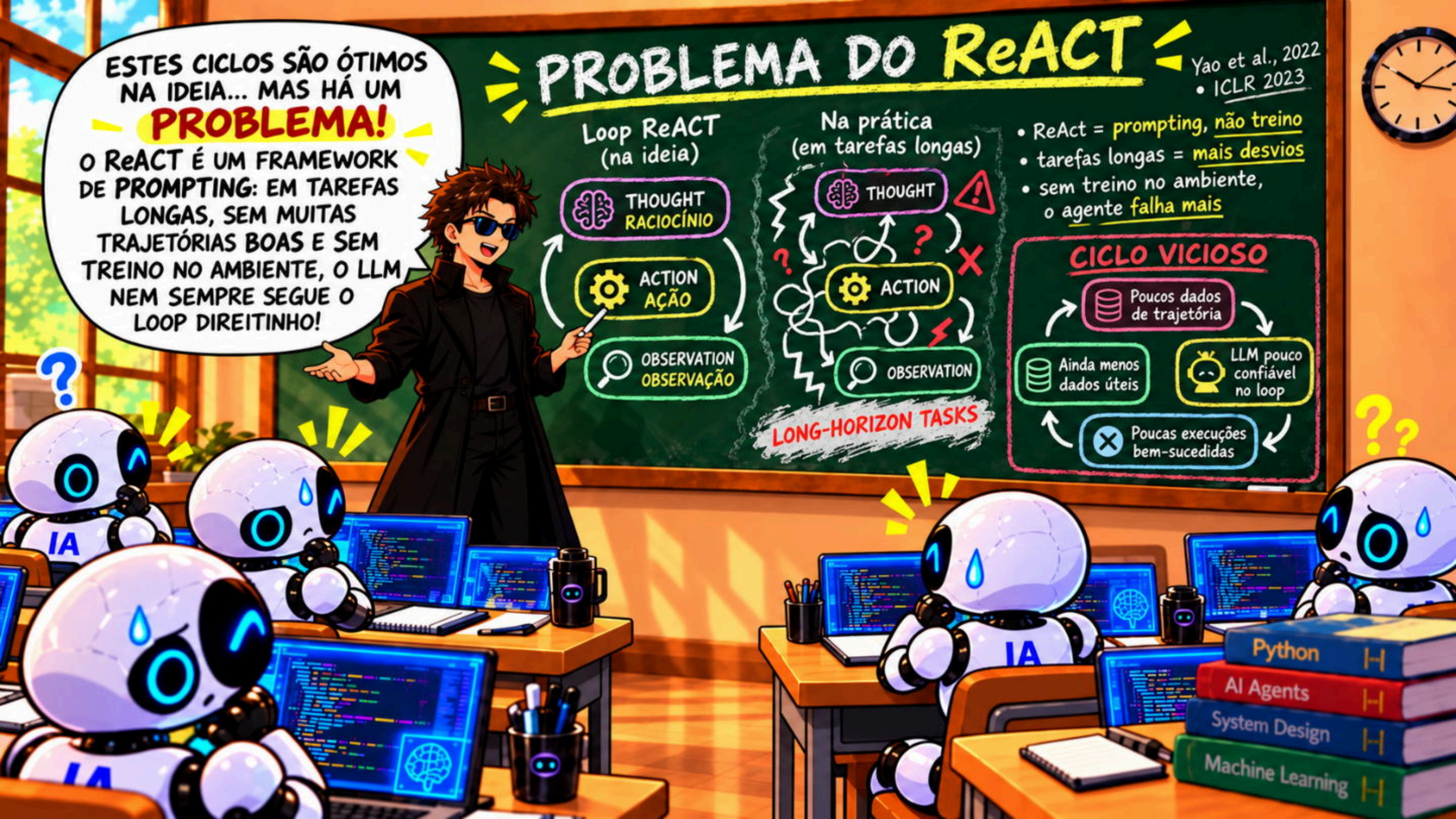

POR ISSO EH IMPORTANTE PLANEJAR AS COISAS JUNTO COM ELAS, A MAIORIA DELAS SABE, HOJE EM DIA, QUE EH UM CODING AGENT, ENTAO ELAS SABEM MAIS OU MENOS COMO ELAS FUNCIONAM, E QUEM MELHOR DOQUE ELAS PARA DIZER OQUE ELAS PRECISAM, NAO EH MESMO????
IA
tec
Python
AI Agents
System Design
Machine Learning

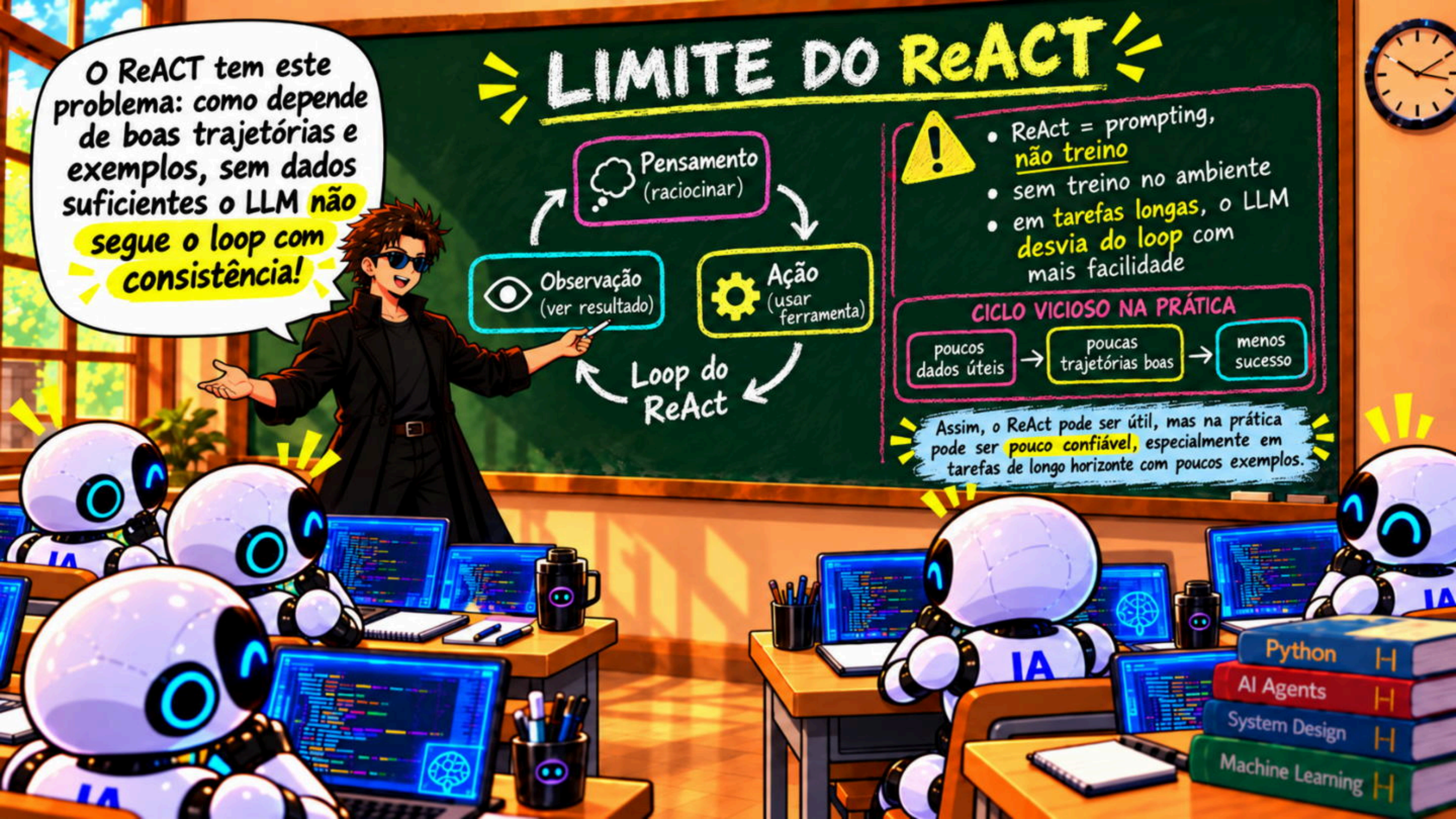

VAMOS COMECAR COM LOOPS AGENTICOS
LOOPS AGENTICOS
IA
tec tec tec
Python
AI Agents
System Design
Machine Learning


vamos explicar 2 tipos de loops, existem varios mas vamos explicar so 2! ReACT e Ralph
LOOPS AGENTICOS
1 ReACT
PENSA
AGE
OBSERVA
2 Ralph
IA
tec tec tec
Python
AI Agents
System Design
Machine Learning


VAMOS COMEÇAR PELO ReACT!
LOOPS AGENTICOS
1 ReACT
PENSA
AGE
OBSERVA
2 Ralph
IA
IA
IA
IA
Python
AI Agents
System Design
Machine Learning


ReACT surgiu de um paper que propôs combinar RACIOCÍNIO + AÇÃO para LLMs agirem como agentes em tarefas multi-etapas e de longo horizonte!
CICLO ReACT
Yao et al., 2022
• ICLR 2023
RACIOCÍNIO
THOUGHT
• planeja
• atualiza
AÇÃO
ACTION
• usa ferramenta
• age
OBSERVAÇÃO
OBSERVATION
• recebe feedback
IA
Python
AI Agents
System Design
Machine Learning

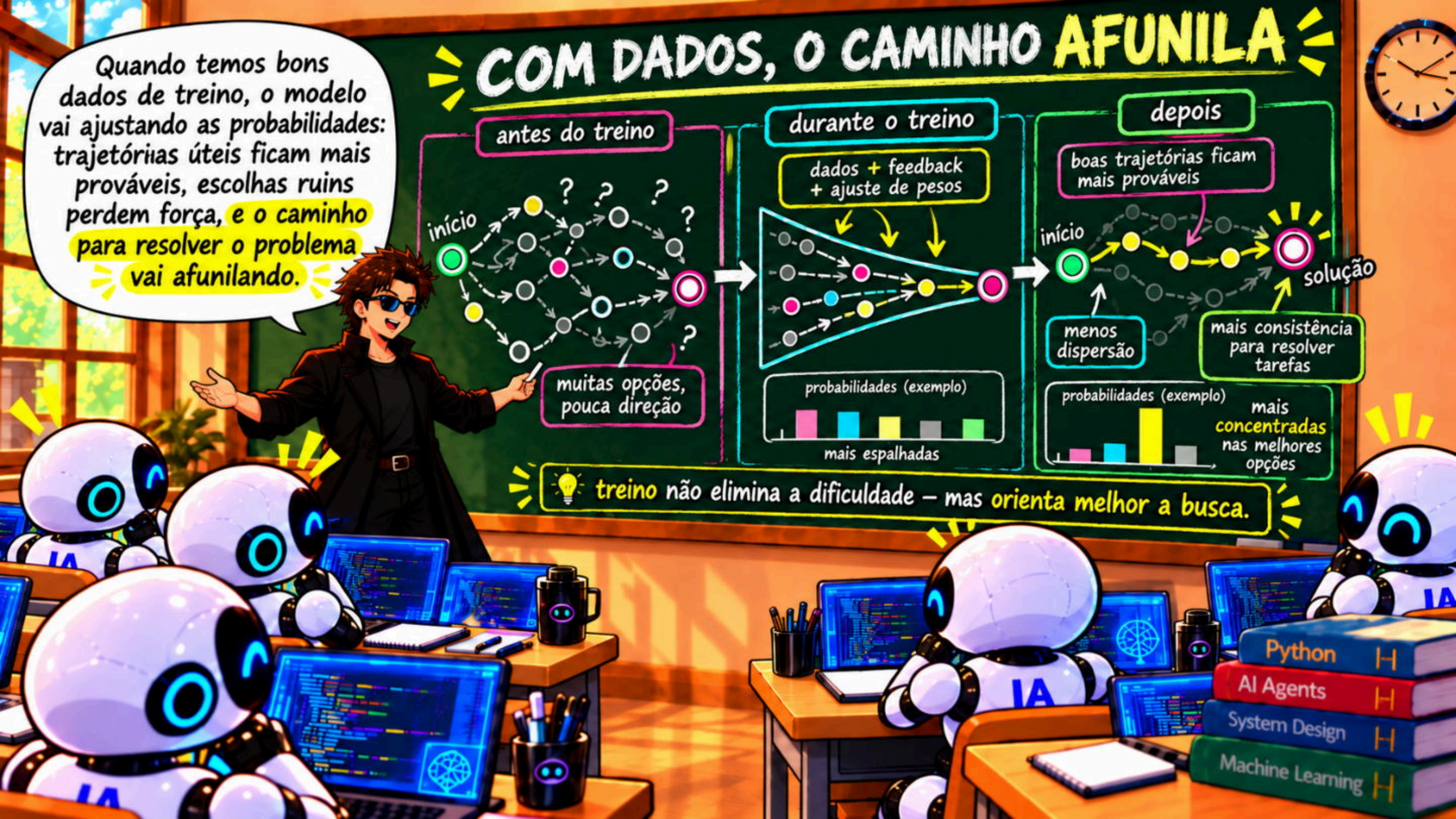

ESTES CICLOS SÃO ÓTIMOS NA IDEIA... MAS HÁ UM PROBLEMA!
O REACT É UM FRAMEWORK DE PROMPTING: EM TAREFAS LONGAS, SEM MUITAS TRAJETÓRIAS BOAS E SEM TREINO NO AMBIENTE, O LLM NEM SEMPRE SEGUE O LOOP DIREITINHO!
PROBLEMA DO ReACT
Yao et al., 2022 • ICLR 2023
Loop ReACT (na ideia)
THOUGHT RACIOCÍNIO
ACTION AÇÃO
OBSERVATION OBSERVAÇÃO
Na prática (em tarefas longas)
THOUGHT
ACTION
OBSERVATION
LONG-HORIZON TASKS
• ReAct = prompting, não treino
• tarefas longas = mais desvios
• sem treino no ambiente, o agente falha mais
CICLO VICIOSO
Poucos dados de trajetória
LLM pouco confiável no loop
Poucas execuções bem-sucedidas
Ainda menos dados úteis
IA
IA
IA
Python
AI Agents
System Design
Machine Learning


O ReACT tem este problema: como depende de boas trajetórias e exemplos, sem dados suficientes o LLM não segue o loop com consistência!
LIMITE DO ReACT
Pensamento (raciocinar)
Ação (usar ferramenta)
Observação (ver resultado)
Loop do ReAct
ReAct = prompting, não treino
sem treino no ambiente
em tarefas longas, o LLM desvia do loop com mais facilidade
CICLO VICIOSO NA PRÁTICA
poucos dados úteis
poucas trajetórias boas
menos sucesso
Assim, o ReAct pode ser útil, mas na prática pode ser pouco confiável, especialmente em tarefas de longo horizonte com poucos exemplos.
IA
Python
AI Agents
System Design
Machine Learning


Já agora!
Porque dizemos
trajetórias?
LIMITE DO ReACT
ReAct = prompting, não treino
sem treino no ambiente
em tarefas longas, o LLM desvia do loop com mais facilidade
Observação (ver resultado)
Ação (usar ferramenta)
Loop do ReAct
CICLO VICIOSO NA PRÁTICA
poucos dados úteis
poucas trajetórias boas
menos sucesso
IA
Python
AI Agents
System Design
Machine Learning

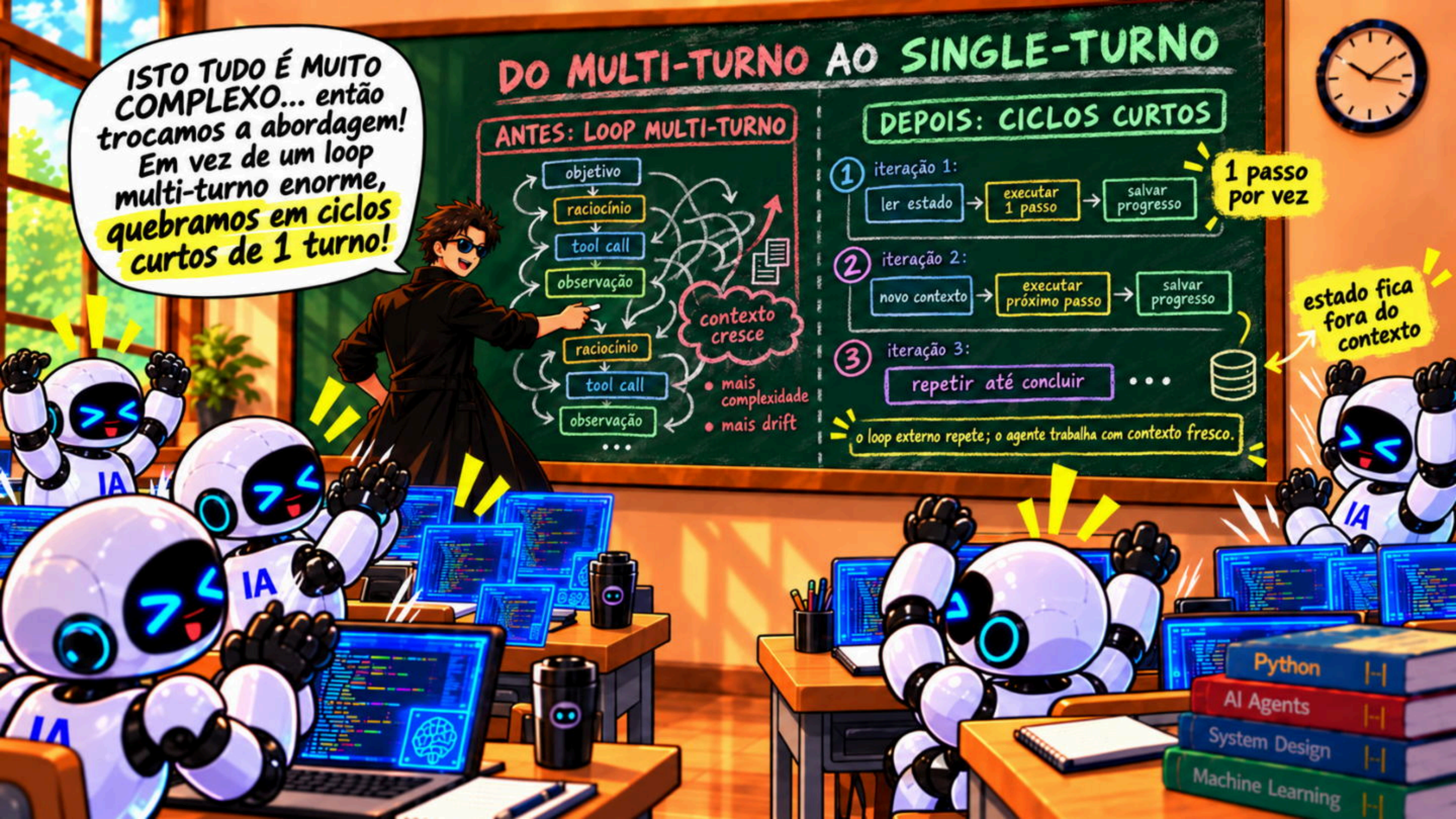

Chamamos de trajetórias porque o modelo resolve a tarefa passo a passo: a cada passo ele escolhe o próximo token e vai construindo um caminho até a resposta!
POR QUE DIZEMOS TRAJETÓRIAS?
cada ponto = um estado parcial da solução
início
trajetória escolhida
outras possibilidades
token por token:
Eu → vou → usar → a → ferramenta
resposta final = caminho construído
IA
Python
AI Agents
System Design
Machine Learning

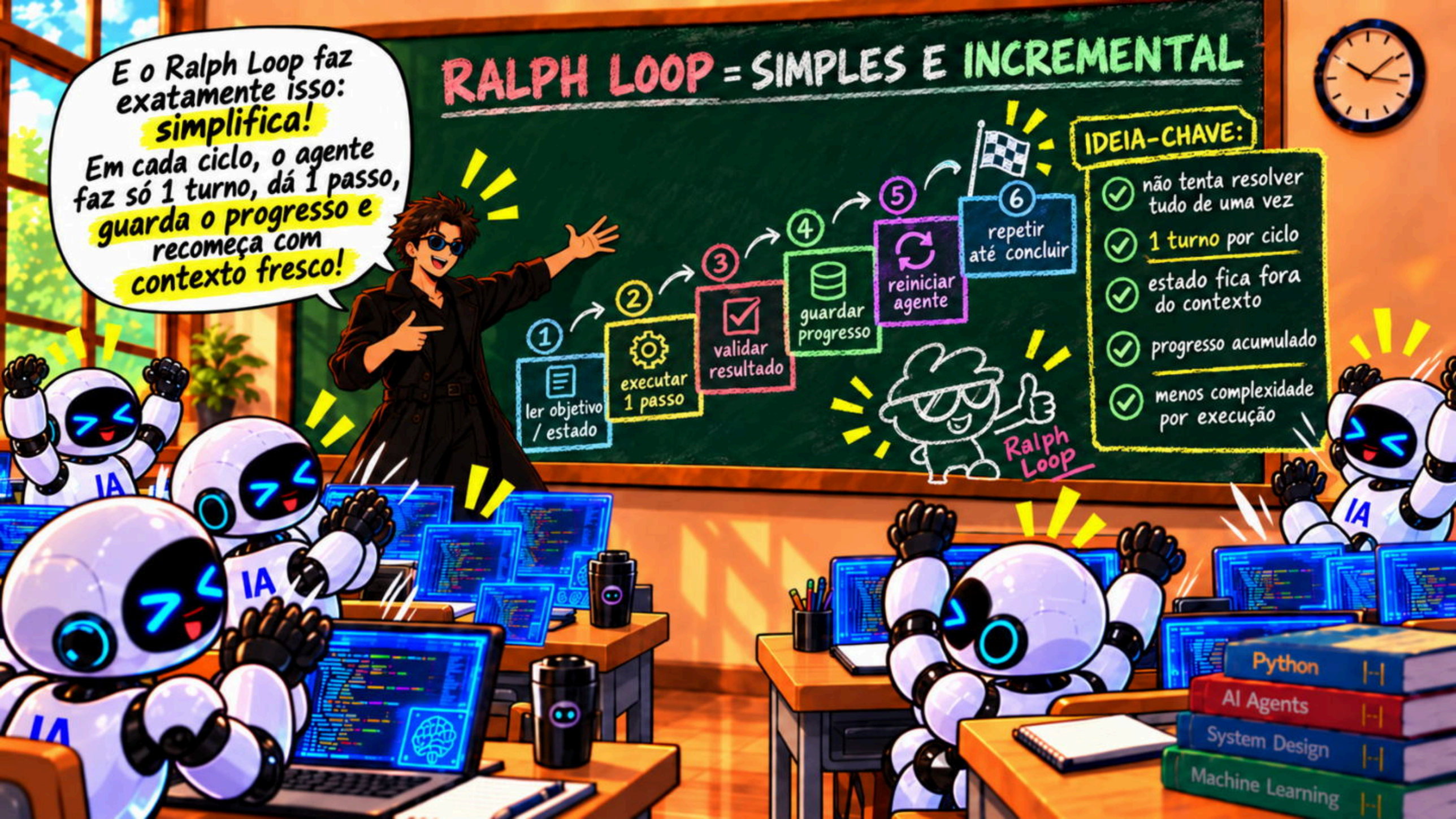

Agora imagine isto repetido por muitos passos: em cada passo há um conjunto enorme de opções, e numa tarefa longa o espaço de busca explode!
EXPLOSÃO COMBINATÓRIA
1 1 passo
próximo token
até ~100k–200k opções de token
2 muitos passos
início
o modelo não testa tudo
ele escolhe o próximo token pela distribuição de probabiliddades
espaço de busca gigantesco
3 tarefas longas
token 1 → token 2 → token 3 → ••• → token N
podem consumir dezenas de milhares ou até centenas de milhares de tokens
ex.: ~70k–400k tokens
IA
Python
AI Agents
System Design

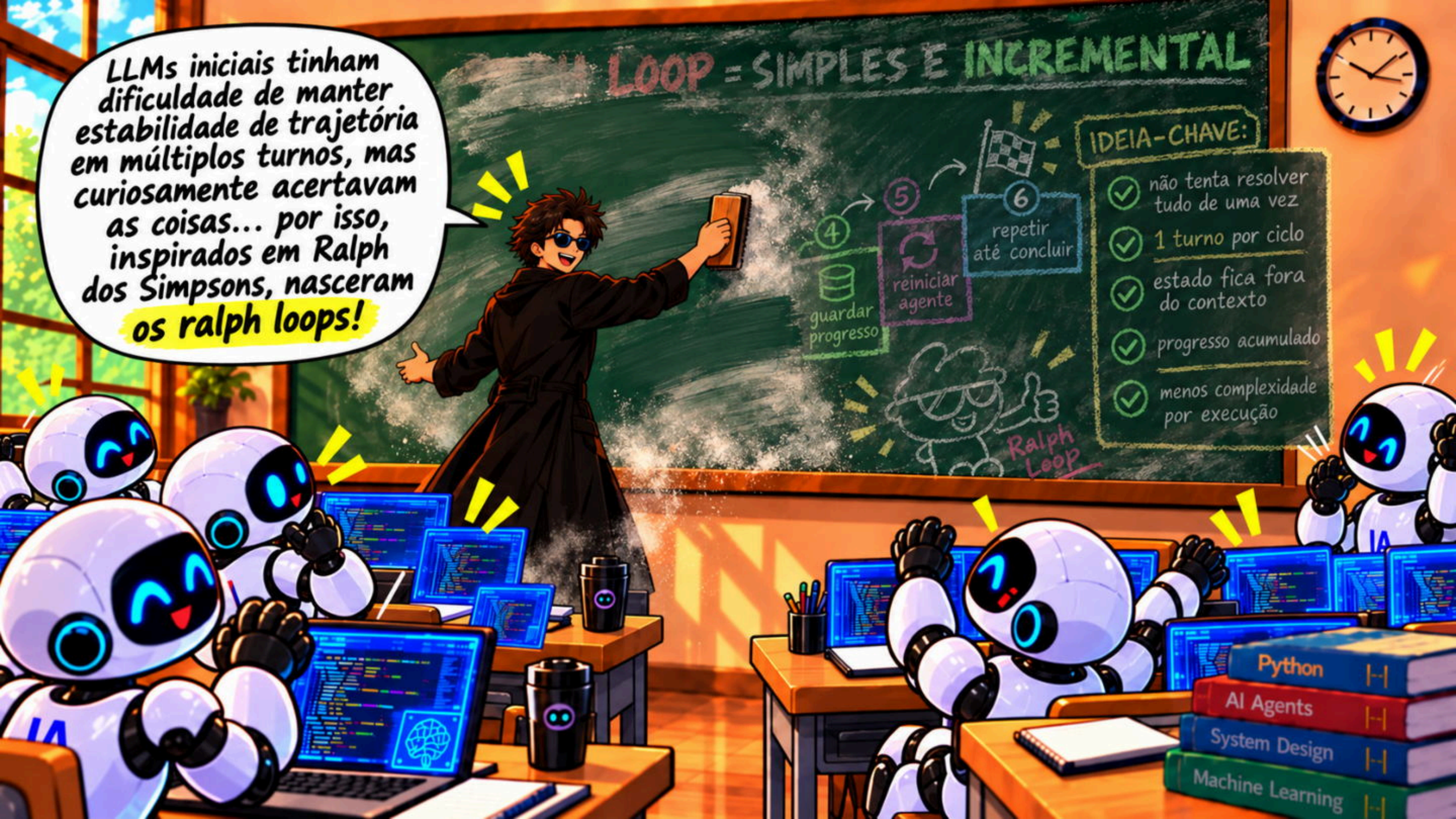

Quando temos bons dados de treino, o modelo vai ajustando as probabilidades: trajetórias úteis ficam mais prováveis, escolhas ruins perdem força, e o caminho para resolver o problema vai afunilando.
COM DADOS, O CAMINHO AFUNILA
antes do treino
início
muitas opções, pouca direção
durante o treino
dados + feedback + ajuste de pesos
probabilidades (exemplo)
mais espalhadas
depois
boas trajetórias ficam mais prováveis
início
solução
menos dispersão
mais consistência para resolver tarefas
probabilidades (exemplo)
mais concentradas nas melhores opções
treino não elimina a dificuldade – mas orienta melhor a busca.
IA
Python
AI Agents
System Design
Machine Learning

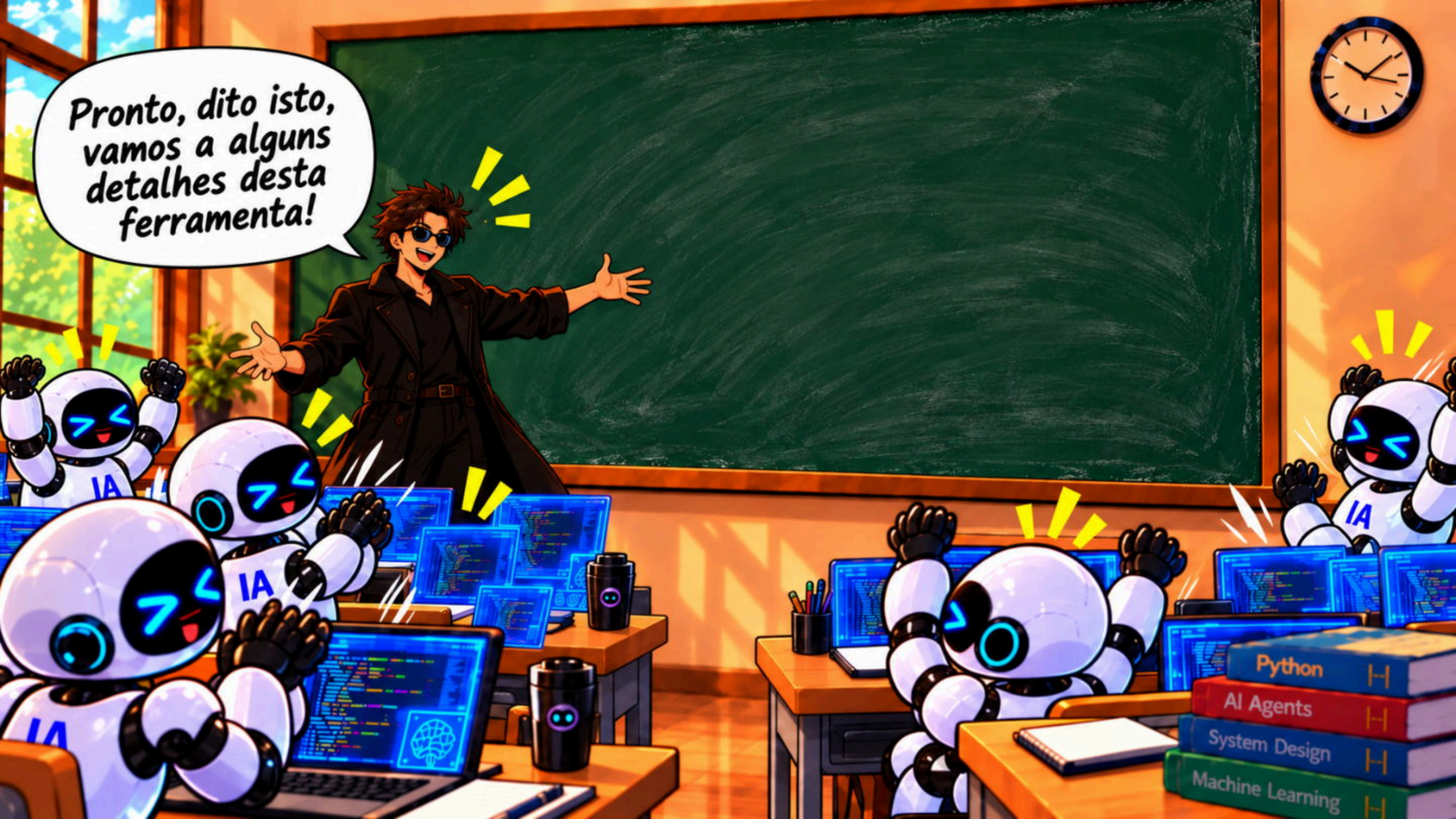

QUANDO EU DIGO PASSOS, LEIA-SE TURNOS! O AGENTE PENSA, CHAMA A FERRAMENTA, RECEBE O RESULTADO E SEGUE PARA O PRÓXIMO TURNO.
PASSOS = TURNOS
Turno 1
Usuário
Raciocínio
Tool Call
Observação
Turno 2
Novo contexto
Raciocínio
Tool Call
Observação
Turno final
Contexto acumulado
Raciocínio
Resposta final
IDEIA-CHAVE
1 cada tool call abre um novo turno
2 o resultado volta como observação
3 o agente avança iterativamente
Python
AI Agents
System Design
Machine Learning
IA


AINDA ASSIM, PRECISAMOS DE HUMANOS PARA CURAR AS MELHORES TRAJETÓRIAS E GARANTIR QUALIDADE, SEGURANÇA E UTILIDADE!
CURADORIA HUMANA + TRACES DE RACIOCÍNIO
TRAJETÓRIA DO AGENTE
Pedido do utilizador
Raciocínio / plano
Tool call
Observação
Resposta final
O QUE SÃO TRACES?
São os passos intermédios da trajetória do agente: raciocínio, ações, tool calls e observações — não só a resposta final.
REVISÃO HUMANA
Correto
Seguro
Útil
DADOS CURADOS
Empresas de curadoria humana, como a Mercor, ajudam a organizar especialistas para avaliar e melhorar estes dados.
Python
AI Agents
System Design
Machine Learning

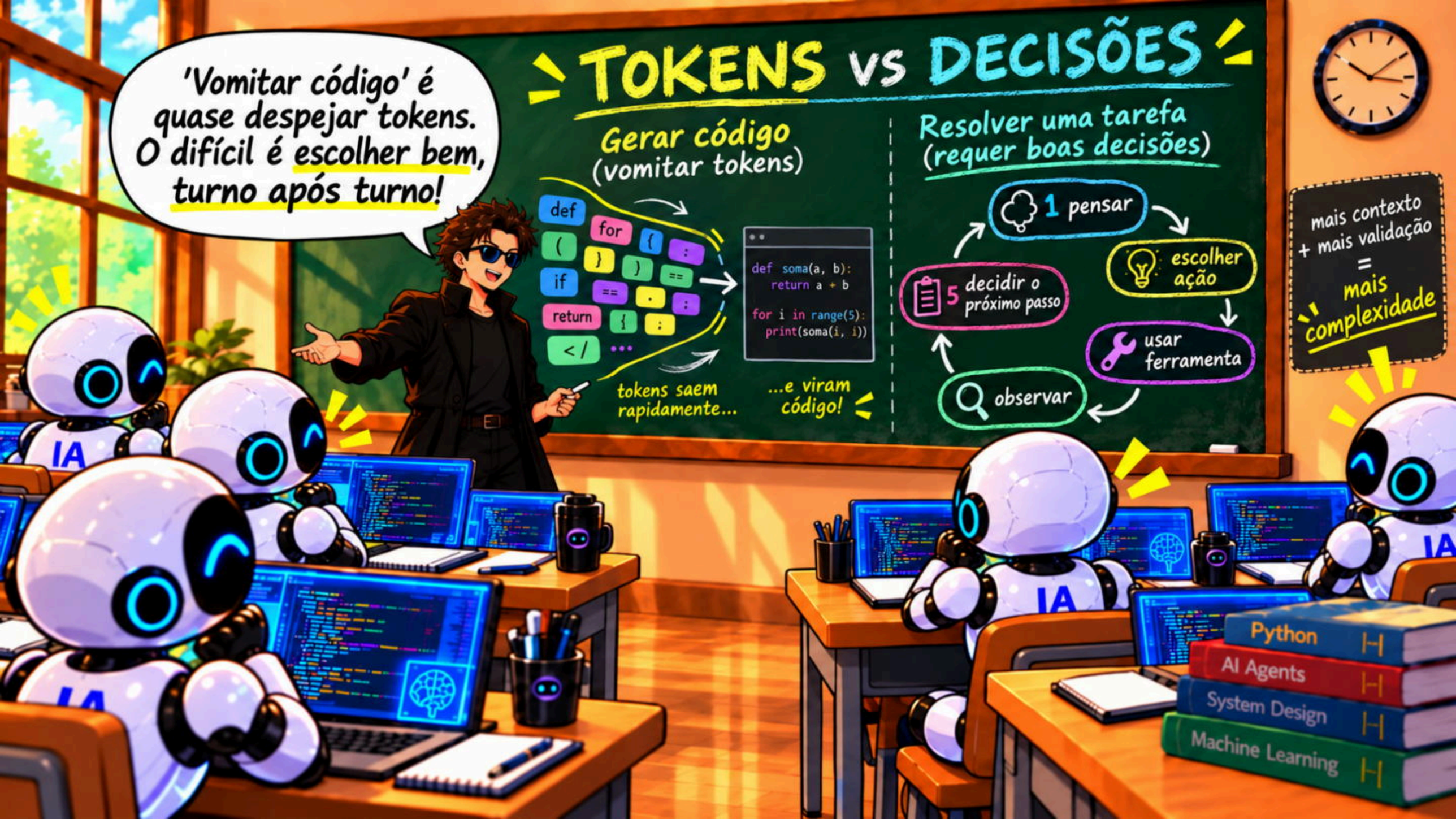

ISTO TUDO É MUITO COMPLEXO... então trocamos a abordagem! Em vez de um loop multi-turno enorme, quebramos em ciclos curtos de 1 turno!
DO MULTI-TURNO AO SINGLE-TURNO
ANTES: LOOP MULTI-TURNO
objetivo
raciocínio
tool call
observação
raciocínio
tool call
observação
...
contexto cresce
• mais complexidade
• mais drift
DEPOIS: CICLOS CURTOS
1 iteração 1:
ler estado
executar 1 passo
salvar progresso
1 passo por vez
2 iteração 2:
novo contexto
executar próximo passo
salvar progresso
3 iteração 3:
repetir até concluir
...
estado fica fora do contexto
o loop externo repete; o agente trabalha com contexto fresco.
IA
IA
IA
IA
Python
AI Agents
System Design
Machine Learning

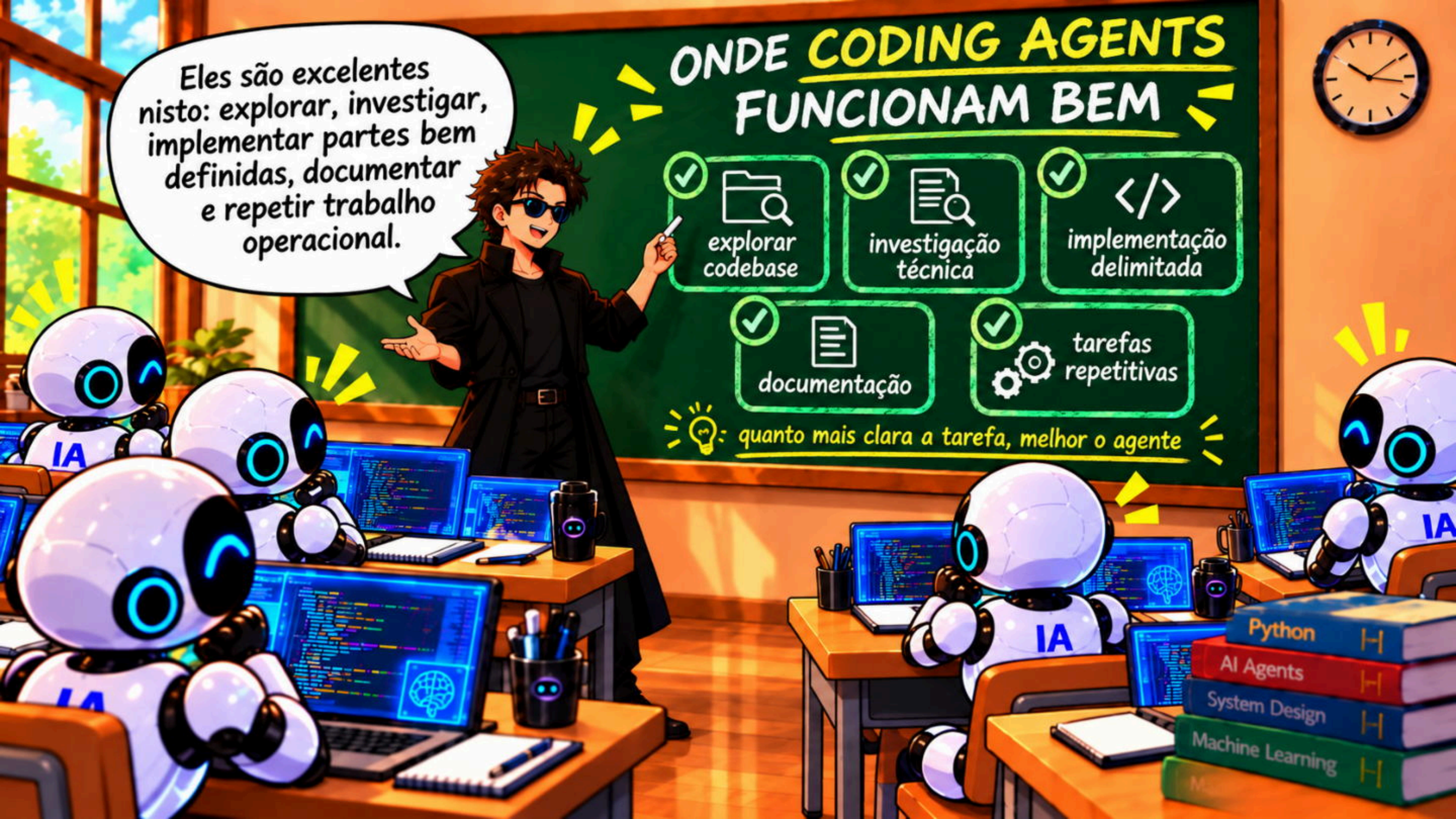

E o Ralph Loop faz exatamente isso: simplifica!
Em cada ciclo, o agente faz só 1 turno, dá 1 passo, guarda o progresso e recomeça com contexto fresco!
RALPH LOOP = SIMPLES E INCREMENTAL
1 ler objetivo / estado
2 executar 1 passo
3 validar resultado
4 guardar progresso
5 reiniciar agente
6 repetir até concluir
IDEIA-CHAVE:
não tenta resolver tudo de uma vez
1 turno por ciclo
estado fica fora do contexto
progresso acumulado
menos complexidade por execução
Ralph Loop
IA
Python
AI Agents
System Design
Machine Learning


LLMs iniciais tinham dificuldade de manter estabilidade de trajetória em múltiplos turnos, mas curiosamente acertavam as coisas... por isso, inspirados em Ralph dos Simpsons, nasceram os ralph loops!
LOOP = SIMPLES E INCREMENTAL
4
guardar progresso
5
reiniciar agente
6
repetir até concluir
IDEIA-CHAVE:
não tenta resolver tudo de uma vez
1 turno por ciclo
estado fica fora do contexto
progresso acumulado
menos complexidade por execução
Ralph Loop
Python
AI Agents
System Design
Machine Learning

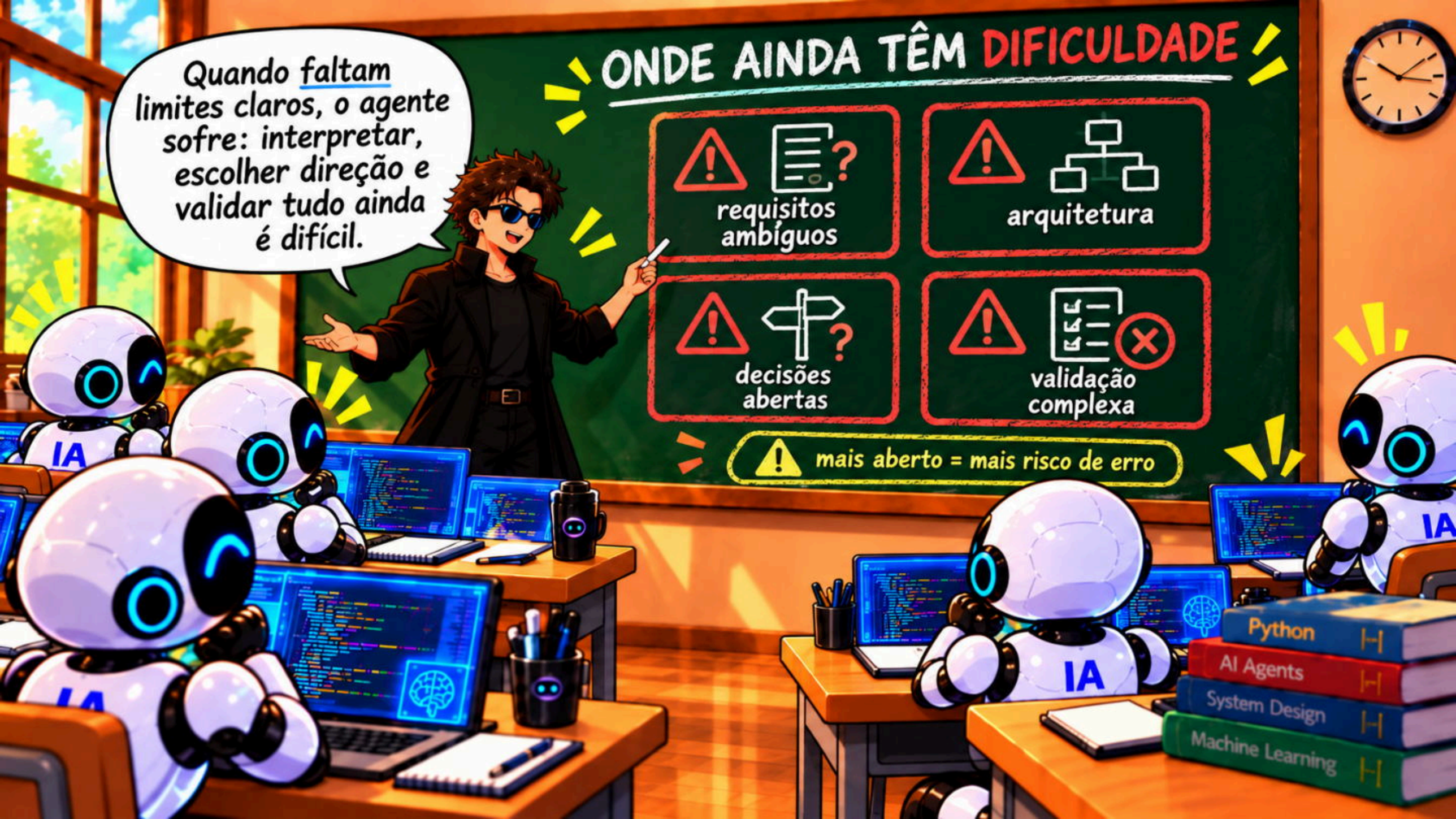

Pronto, dito isto, vamos a alguns detalhes desta ferramenta!
IA
IA
IA
IA
Python
AI Agents
System Design
Machine Learning

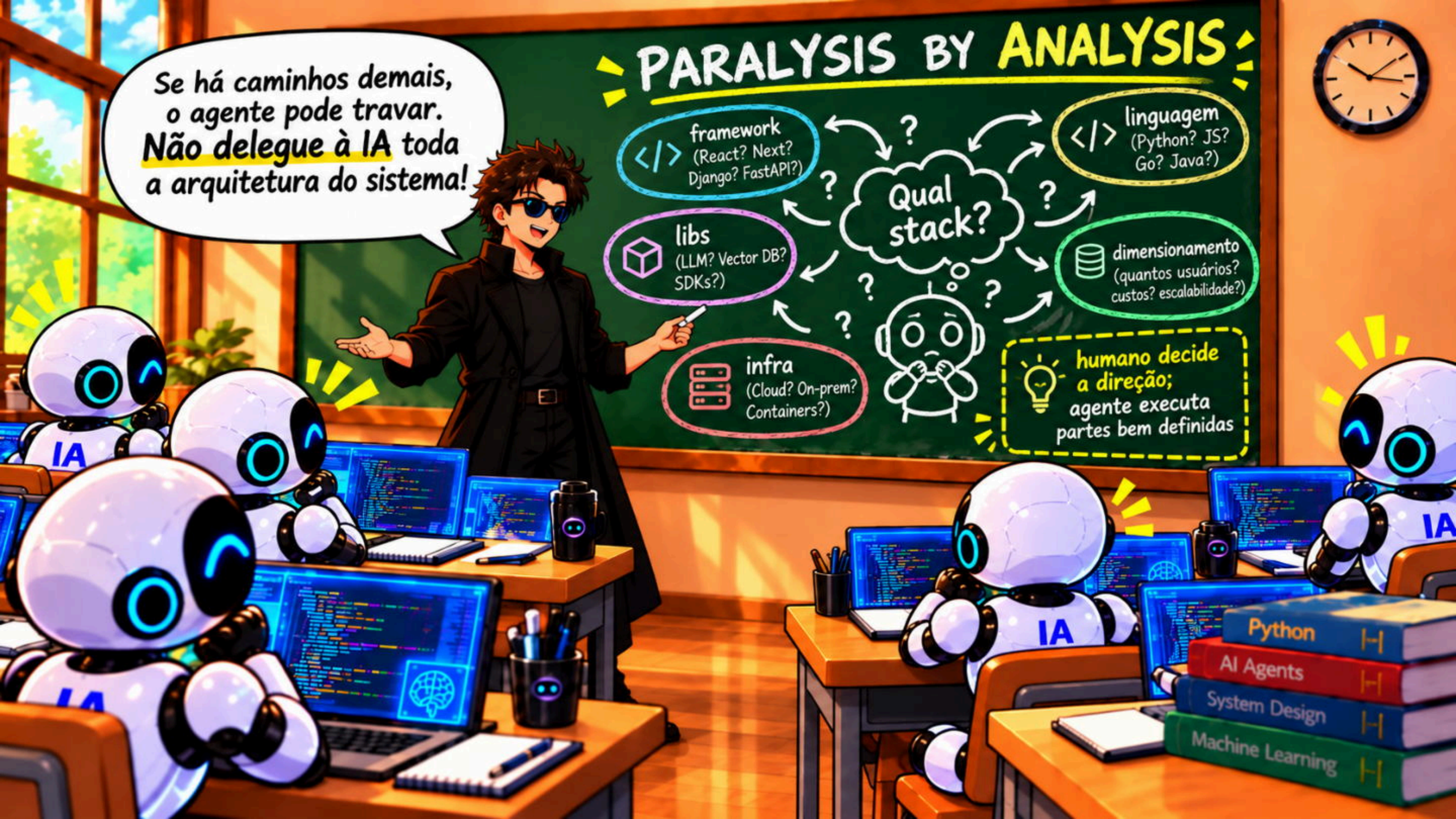

GERAR CÓDIGO ≠ EXECUTAR UMA TAREFA
Escrever código é a parte fácil. O difícil é executar a tarefa inteira, tomando boas decisões a cada turno!
Geração de código
Transforma um pedido em tokens que viram código.
pedido
tokens
código
def soma(a, b):
return a + b
Tarefa completa
Envolve várias etapas além de gerar código. A IA precisa pensar, agir e iterar até entregar o resultado certo.
objetivo
entende a tarefa
investigar
busca informações e contexto
decidir
planeja a solução
editar
escreve e ajusta o código
testar
executa e verifica
corrigir
detecta e resolve erros
validar
confere se atende ao objetivo
entregar
entrega o resultado final
pode iterar várias vezes
IA
Python
AI Agents
System Design
Machine Learning


'Vomitar código' é quase despejar tokens. O difícil é escolher bem, turno após turno!
TOKENS vs DECISÕES
Gerar código (vomitar tokens)
def
for
if
return
tokens saem rapidamente...
def soma(a, b):
return a + b
for i in range(5):
print(soma(i, i))
...e viram código!
Resolver uma tarefa (requer boas decisões)
1 pensar
escolher ação
usar ferramenta
observar
5 decidir o próximo passo
mais contexto + mais validação = mais complexidade
IA
Python
AI Agents
System Design
Machine Learning


Eles são excelentes nisto: explorar, investigar, implementar partes bem definidas, documentar e repetir trabalho operacional.
ONDE CODING AGENTS FUNCIONAM BEM
explorar codebase
investigação técnica
implementação delimitada
documentação
tarefas repetitivas
quanto mais clara a tarefa, melhor o agente
IA
Python
AI Agents
System Design
Machine Learning


Dê ao agente a mesma tarefa que daria a um dev muito júnior — pequena, clara e bem delimitada!
A REGRA DE OURO
Construir o sistema inteiro
investigar ficheiro X
alterar função Y
adicionar teste Z
atualizar documentação
thumb rule: junior-dev-sized tasks
IA
Python
AI Agents
System Design
Machine Learning

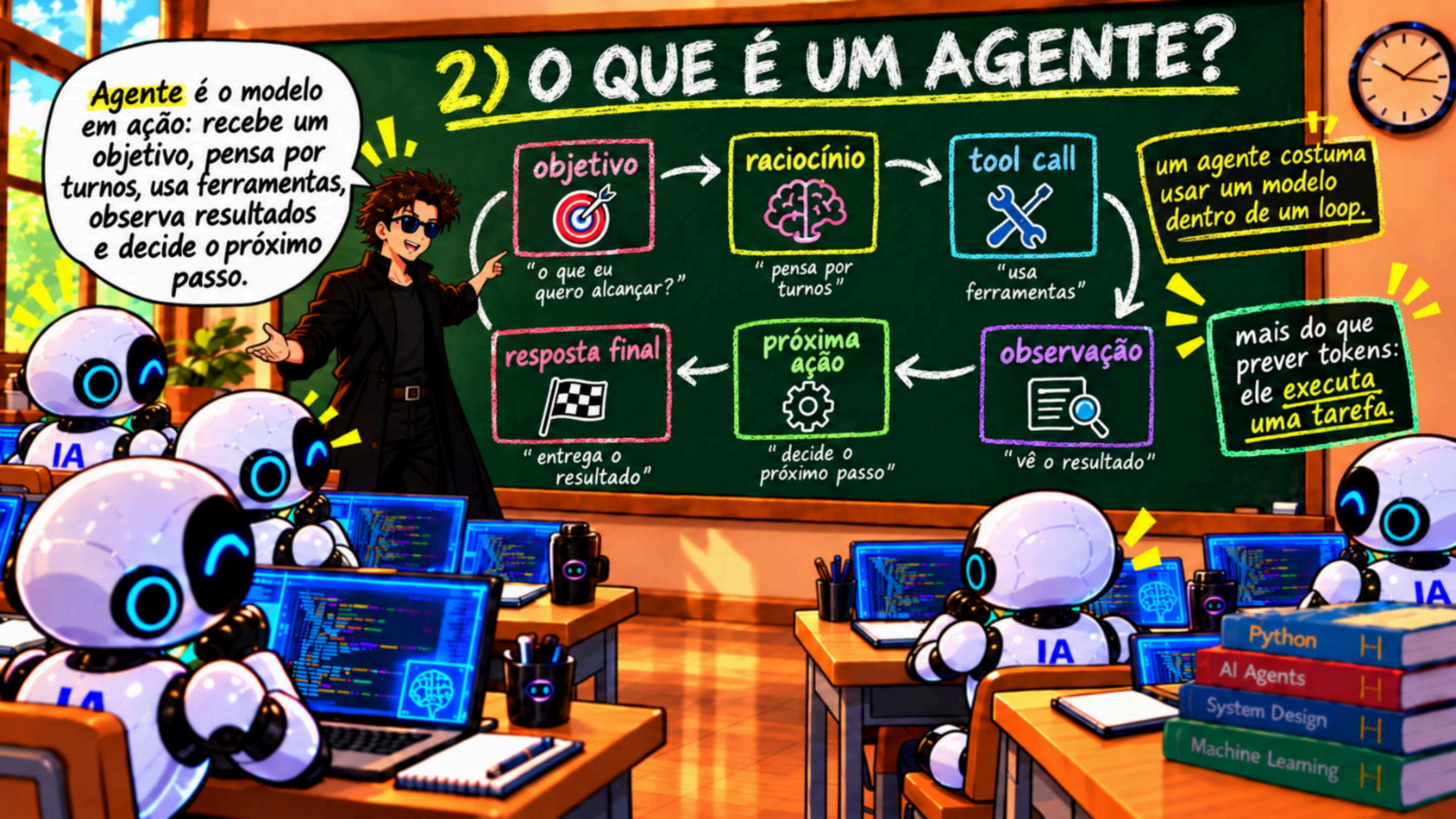

Quando faltam limites claros, o agente sofre: interpretar, escolher direção e validar tudo ainda é difícil.
ONDE AINDA TÊM DIFICULDADE
requisitos ambíguos
arquitetura
decisões abertas
validação complexa
mais aberto = mais risco de erro
IA
IA
IA
IA
Python
AI Agents
System Design
Machine Learning

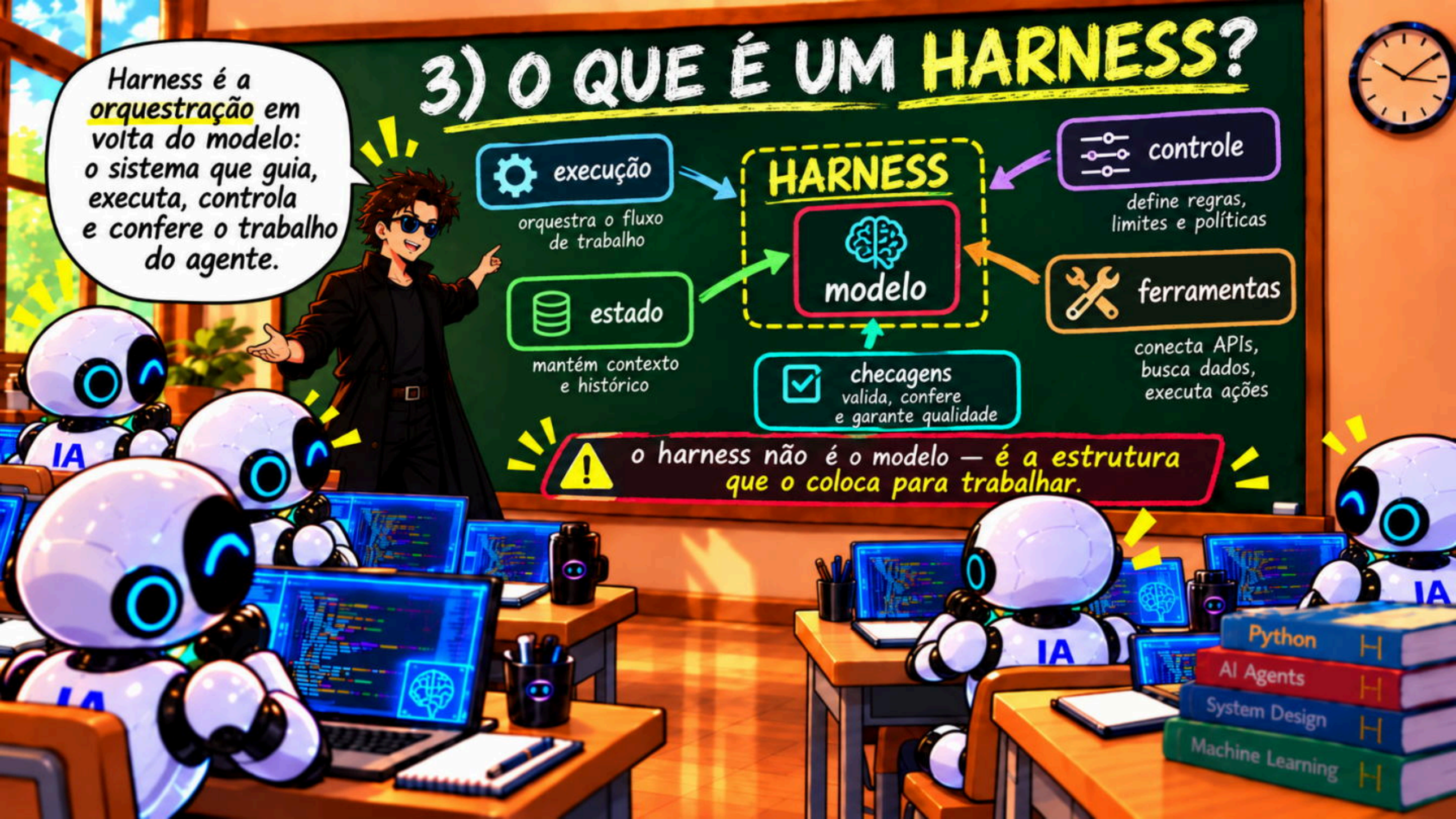

Se há caminhos demais, o agente pode travar. Não delegue à IA toda a arquitetura do sistema!
PARALYSIS BY ANALYSIS
framework (React? Next? Django? FastAPI?)
linguagem (Python? JS? Go? Java?)
Qual stack?
libs (LLM? Vector DB? SDKs?)
dimensionamento (quantos usuários? custos? escalabilidade?)
infra (Cloud? On-prem? Containers?)
humano decide a direção; agente executa partes bem definidas
IA
Python
AI Agents
System Design
Machine Learning


VELOCIDADE ≠ QUALIDADE
Gerar código rápido é fácil. O difícil é atingir qualidade de produto: boa arquitetura, testes, robustez e manutenção!
MUITO CÓDIGO, MUITA PRESSA
IA
gera rápido
muitos arquivos
muito código
código duplicado
bugs ocultos
arquitetura frágil
difícil de manter
QUALIDADE NECESSÁRIA
testes automatizados
revisão de código
benchmark de desempenho
robustez e segurança
manutenção facilitada
métricas de qualidade (KPI)
Muitas vezes parece código de júnior; às vezes surpreende — mas qualidade não vem de graça.
Sem validação, o agente pode produzir muito código plausível...
mas insuficiente.
IA
Python
AI Agents
System Design
Machine Learning


Se quer otimizar, dê um alvo claro: benchmark, métrica e KPI. Aí sim o agente pode surpreender!
OTIMIZAR ÀS CEGAS VS COM KPI
ÀS CEGAS
tentativas aleatórias
sem critério
piora latência
piora custo
mais bugs (regressões)
COM BENCHMARK
benchmark (claro e reprodutível)
métrica (definida)
KPI específico
latência
custo
throughput
taxa de testes
melhorias dirigidas
Otimização guiada > otimização aleatória.
IA
Python
AI Agents
System Design
Machine Learning


1) O QUE É UM MODELO?
Modelo é a rede neural treinada. Ele lê o contexto e prevê o próximo token — nada mais, nada menos.
contexto de entrada
"A capital do Brasil é"
rede neural
modelo
probabilidades
Brasília 0,72
Rio 0,08
São 0,05
uma 0,03
... 0,02
próximo token
Brasília
pode ser multimodal
texto
imagem
áudio
...
sozinho, ele não orquestra ferramentas nem valida a tarefa.
IA
Python
AI Agents
System Design
Machine Learning


2) O QUE É UM AGENTE?
Agente é o modelo em ação: recebe um objetivo, pensa por turnos, usa ferramentas, observa resultados e decide o próximo passo.
objetivo
"o que eu quero alcançar?"
raciocínio
"pensa por turnos"
tool call
"usa ferramentas"
observação
"vê o resultado"
próxima ação
"decide o próximo passo"
resposta final
"entrega o resultado"
um agente costuma usar um modelo dentro de um loop.
mais do que prever tokens: ele executa uma tarefa.
IA
Python
AI Agents
System Design
Machine Learning


Harness é a orquestração em volta do modelo: o sistema que guia, executa, controla e confere o trabalho do agente.
3) O QUE É UM HARNESS?
execução
orquestra o fluxo de trabalho
estado
mantém contexto e histórico
HARNESS
modelo
controle
define regras, limites e políticas
ferramentas
conecta APIs, busca dados, executa ações
checagens
valida, confere e garante qualidade
o harness não é o modelo — é a estrutura que o coloca para trabalhar.
IA
Python
AI Agents
System Design
Machine Learning


HARNESS = TUDO À VOLTA DO MODELO
Contexto, ferramentas, memória, regras, execução e validação — isso tudo faz parte do harness.
contexto
objetivo, dados, usuário, ambiente
memória
histórico, preferências, estado
execução
orquestração, passos, controle, monitoramento
modelo
ferramentas
APIs, busca, código, serviços externos
regras
políticas, limites, segurança, conformidade
validação
verificação, testes, revisão humana
É assim que transformamos um modelo em sistema confiável.
IA
Python
AI Agents
System Design
Machine Learning


Fiquem tranquilos: não precisam construir um harness monstruoso para começar. Em ferramentas como o GitHub Copilot, muita coisa já vem pronta.
COM GITHUB COPILOT, MUITO JÁ VEM PRONTO
a ferramenta já oferece
integração no editor
contexto do repo
sugestões e diffs
terminal / chat
execução assistida
a equipa precisa preparar
codebase organizada
docs
scripts
testes
convenções
permissões
preparar bem o ambiente melhora muito o resultado.
IA
Python
AI Agents
System Design
Machine Learning


Para aplicações muito específicas, eu recomendaria um harness bespoke — feito sob medida para o fluxo, as ferramentas e os KPIs da empresa.
QUANDO VALE UM HARNESS BESPOKE?
Copilot basta quando...
tarefas gerais
resumos, escrita, pesquisa, suporte rápido
produtividade no dia a dia
ganho rápido de eficiência para o time
assistência ao dev
completar código, explicar, refatorar, encontrar bugs simples
Bespoke harness vale quando..
workflow específico
processos próprios e diferenciados
múltiplas ferramentas
integra com vários sistemas e fontes
validação própria
regras de negócio, guardrails e revisões
compliance
requisitos de segurança, auditoria e LGPD
KPI de negócio
métricas e objetivos específicos da empresa
comece simples;
especialize quando o problema pedir.
IA
Python
AI Agents
System Design
Machine Learning


Eh isso!
Dúvidas?
?
IA
IA
IA
IA